Smart Factory Sensor Analytics

Create the Data Simulate sensor data for 5 machines with 24 sensors and 60 minutes using NumPy, and check basic array properties.

1. Introduce & Fix Errors Randomly mark 5% values as missing (-1) and replace them with the average of the same sensor.
2. Normalize the Data Scale sensor values between 0 and 1 using min–max normalization.
3. Analyze the Data Calculate averages, standard deviations, maximums, and identify which machine and sensor show the highest values.
4. Detect Problems & Reshape Find anomalies using statistical rules, count them per machine, and reshape data for further analysis.

Step 1: Creating Sensor Data using Numpy

```
import numpy as np
from loguru import logger
```

```
# Given Parameters
machines = 5
sensors = 24
minutes = 60
```

```
data = np.random.randint(10, 100, size = (machines, sensors, minutes))
logger.info(f"Shape of data:, {data.shape}")
logger.info(f"Types of data:, {data.dtype}")
logger.info(f"Maximum value of data:, {data.max()}")
logger.info(f"Maximum value of data:, {data.min()}")
```

```
2026-02-18 09:50:08.878 | INFO     | __main__:<cell line: 0>:2 - Shape of data:, (5, 24, 60)
2026-02-18 09:50:08.879 | INFO     | __main__:<cell line: 0>:3 - Types of data:, int64
2026-02-18 09:50:08.880 | INFO     | __main__:<cell line: 0>:4 - Maximum value of data:, 99
2026-02-18 09:50:08.882 | INFO     | __main__:<cell line: 0>:5 - Maximum value of data:, 10
```

Step 2: Fixing Errors

```
mask = np.random.rand(*data.shape) < 0.5
data[mask] = -1

for m in range(machines):
  for s in range(sensors):
    sensor_values = data[m, s, :]
    missing = sensor_values == -1
    mean_val = sensor_values[~missing].mean()
    sensor_values[missing] = mean_val
    data [m, s, :] = sensor_values

logger.info(f"Sensor values: {sensor_values}")
```

```
2026-02-18 10:02:48.084 | INFO     | __main__:<cell line: 0>:12 - Sensor values: [58 58 39 60 58 58 60 60 58 58 58 58 58 64 59 5
 58 60 58 60 58 59 58 58 60 58 58 60 60 58 58 58 55 58 60 58 60 58 60 58
 58 58 63 52 58 60 58 58 60 60 55 60]
```

Step 3: Normalize the Data

Scale values between 0 and 1 using min–max normalization.

```
min_val = np.min(data)
max_val = np.max(data)
normal_data = (data - min_val) / (max_val - min_val)
logger.info(f"Normalized data: {normal_data}")
```

```
2026-02-18 10:07:25.620 | INFO     | __main__:<cell line: 0>:4 - Normalized data: [[[0.47191011 0.47191011 0.47191011 ... 0.4719
  [0.53932584 0.5505618  0.52808989 ... 0.52808989 0.53932584 0.53932584]
  [0.5505618  0.5505618  0.5505618  ... 0.56179775 0.56179775 0.5505618 ]
  ...
  [0.52808989 0.49438202 0.49438202 ... 0.49438202 0.49438202 0.49438202]
  [0.48314607 0.48314607 0.48314607 ... 0.48314607 0.48314607 0.48314607]
```

```
 [0.52808989 0.52808989 0.52808989 ... 0.47191011 0.52808989 0.52808989]]

[[0.51685393 0.52808989 0.50561798 ... 0.51685393 0.50561798 0.51685393]
 [0.51685393 0.49438202 0.49438202 ... 0.49438202 0.49438202 0.51685393]
 [0.50561798 0.51685393 0.51685393 ... 0.50561798 0.50561798 0.50561798]
 ...
 [0.47191011 0.96629213 0.47191011 ... 0.46067416 0.47191011 0.46067416]
 [0.51685393 0.51685393 0.51685393 ... 0.51685393 0.51685393 0.60674157]
 [0.59550562 0.59550562 0.59550562 ... 0.59550562 0.59550562 0.59550562]]

[[0.46067416 0.49438202 0.49438202 ... 0.49438202 0.49438202 0.46067416]
 [0.53932584 0.53932584 0.53932584 ... 0.53932584 0.53932584 0.53932584]
 [0.51685393 0.51685393 0.60674157 ... 0.5505618  0.51685393 0.51685393]
 ...
 [0.46067416 0.46067416 0.46067416 ... 0.52808989 0.46067416 0.46067416]
 [0.43820225 0.43820225 0.46067416 ... 0.46067416 0.41573034 0.46067416]
 [0.52808989 0.52808989 0.52808989 ... 0.52808989 0.52808989 0.52808989]]

[[0.58426966 0.58426966 0.57303371 ... 0.57303371 0.5505618  0.57303371]
 [0.58426966 0.58426966 0.58426966 ... 0.58426966 0.58426966 0.58426966]
 [0.38202247 0.40449438 0.38202247 ... 0.38202247 0.38202247 0.37078652]
 ...
 [0.4494382  0.4494382  0.4494382  ... 0.47191011 0.46067416 0.46067416]
 [0.48314607 0.49438202 0.49438202 ... 0.49438202 0.49438202 0.49438202]
 [0.48314607 0.48314607 0.53932584 ... 0.51685393 0.48314607 0.51685393]]

[[0.43820225 0.42696629 0.43820225 ... 0.43820225 0.43820225 0.42696629]
 [0.58426966 0.53932584 0.62921348 ... 0.83146067 0.53932584 0.53932584]
 [0.49438202 0.49438202 0.15730337 ... 0.49438202 0.50561798 0.49438202]
 ...
 [0.46067416 0.48314607 0.46067416 ... 0.48314607 0.48314607 0.47191011]
 [0.51685393 0.51685393 0.51685393 ... 0.51685393 0.51685393 0.51685393]
 [0.53932584 0.53932584 0.3258427  ... 0.56179775 0.50561798 0.56179775]]]
```

Step 4: Analyzing data

```
average_values = np.mean(normal_data, axis = 2)
standard_deviation_values = np.std(normal_data, axis = 2)
maximum_values = np.max(normal_data, axis = 2)

# Highest sensor reading
machine_idx, sensor_idx = np.unravel_index(np.argmax(maximum_values), maximum_values.shape)
logger.info(f"Highest value at Machine {machine_idx}, Sensor {sensor_idx}")
```

```
2026-02-18 10:15:57.358 | INFO     | __main__:<cell line: 0>:7 - Highest value at Machine 0, Sensor 23
```

Step 5: Detect Problems & Reshape

```
compute_threshold = np.mean(normal_data, axis = 2) + 2 * np.std(normal_data, axis = 2) # expands thresholds to match the 3D sha
anomalies_in_data = normal_data > compute_threshold[:, :, None] # boolean mask showing where readings exceed thresholds.
anomaly_counts = np.sum(anomalies_in_data, axis = (1, 2)) # counts anomalies per machine.
logger.info(f"Anomaly counts per machine: {anomaly_counts}")
machine_reshaping = normal_data.shape[0]
reshaped_data = normal_data.reshape((machine_reshaping, -1))
logger.info(f"Reshaped data: {reshaped_data}")
```

```
2026-02-18 10:25:11.742 | INFO     | __main__:<cell line: 0>:4 - Anomaly counts per machine: [37 38 38 30 24]
2026-02-18 10:25:11.745 | INFO     | __main__:<cell line: 0>:7 - Reshaped data: [[0.47191011 0.47191011 0.47191011 ... 0.4719101
 [0.51685393 0.52808989 0.50561798 ... 0.59550562 0.59550562 0.59550562]
 [0.46067416 0.49438202 0.49438202 ... 0.52808989 0.52808989 0.52808989]
 [0.58426966 0.58426966 0.57303371 ... 0.51685393 0.48314607 0.51685393]
 [0.43820225 0.42696629 0.43820225 ... 0.56179775 0.50561798 0.56179775]]
```

Summary

1. We made sensor data for 5 machines, 24 sensors, and 60 minutes, with values between 10 and 99.
2. Then added errors by marking half the values as -1, then fixed them by replacing with the average of that sensor's other readings.
3. Then scaled all sensor values between 0 and 1 so they are easier to compare.
4. After scaling all sensor value calculated averages, standard deviations, and maximums, and found the highest reading at Machine 0, Sensor 23.
5. Finally checked for unusual values (anomalies), counted them per machine [37, 38, 38, 30, 24], and reshaped the data for further study.

Start coding or generate with AI.